AUTHENSOR ALPHA MASTER DOCUMENT
Single-file context for maintaining continuity across the Authensor Alpha monorepo
(“Stripe for AI agents” — governed, deterministic, auditable action execution)

Last updated: 2025-12 (Alpha)

==
0) What Authensor is (in one paragraph)
==

Authensor is a “decision gate” and execution control-plane for agent actions. Agents don’t directly call
Stripe/GitHub/HTTP (or any other connector). They submit an Action Envelope to Authensor, which evaluates
it against an active policy (org/env scoped), records a Receipt, and then enforces deterministic execution
through a claim gate, approval workflow, hardened integrations, and an audit trail (“paper trail”) that
is easy to inspect (receipt viewer) and monitor (metrics + insights).

The alpha focuses on:
- Rock-solid determinism (no accidental double execution)
- Safety-by-default integration hardening
- Policy/approval credibility (versioning + active pointers + state machine)
- Stripe-like DX (quickstart prints receipt link; viewer explains what happened)

==
1) The non-negotiables (alpha invariants)
==

These are the foundations. Do NOT drift from them.

1.1 Schema-first, language-neutral objects
- JSON Schemas in packages/schemas are the SINGLE SOURCE OF TRUTH.
- TS and Python types/models are generated from schemas; never hand-edit generated files.
- If a type changes, change schema first, regenerate, then update code.

1.2 Deterministic execution is mandatory
- Every executable receipt must be claimed before execution.
- Only the claimer can finalize (PATCH executed/failed) via claimId.

- Claims have TTL and can be reclaimed after expiry.
- Already executed receipts must short-circuit (no upstream call).

1.3 Safety-by-default integrations
- HTTP: SSRF guardrails; redirects blocked; private/loopback/link-local blocked; optional plain HTTP gated.
- GitHub: allowlists (deny-by-default in prod when unset), token redaction, rate limit mapping to RATE_LIMITED.
- Stripe: live-mode gated, bounds/allowlists enforced, ALL writes idempotent (Idempotency-Key = receiptId).

1.4 Receipts are the product artifact
- Receipts are persisted in Postgres and are the central debugging surface.
- Receipt viewer must be safe (redaction + HTML escaping + proxy origin gating).
- Receipts must never leak secrets.

1.5 Control-plane credibility
- Policies are persisted, versioned, and selected via active pointer (org/env scoped).
- Approvals are real endpoints; state machine blocks execution until approved.
- State transitions must be enforced server-side and covered by tests.

==========
2) Repo layout and “who owns what”
==========

Monorepo structure (conceptual):

```
/ (root)
  package.json / pnpm-workspace.yaml / turbo.json / Makefile / docker-compose.yml
  .env.example
  alpha_onboarding.md
  CHANGELOG.md

/packages
  schemas/        # JSON schema source of truth
  engine/         # pure evaluation logic (policy -> decision)
  control-plane/  # HTTP API + Postgres persistence + policy/receipts/approvals/metrics
  mcp-server/     # hardened tool executors (http/github/stripe) using claim-gated flow
  sdk-ts/         # TypeScript SDK (devs)
  sdk-py/         # Python SDK (agent builders)

/examples
```

node-quickstart/
python-quickstart/ (if present)

Ownership boundaries:
- schemas: define envelope/receipt/policy contracts (language-neutral).
- engine: policy semantics; SHOULD NOT depend on infra or network.
- control-plane: source of truth for policies/receipts/approvals/claims/metrics; enforces invariants.
- mcp-server: executes tools safely; MUST respect control-plane decisions and claim gating.
- SDKs: ergonomic wrappers; MUST NOT weaken invariants (no “shortcuts” around claims/approvals).

==
3) Terminology (avoid confusion)
==

- Action Envelope: the request from agent → Authensor describing an intended action.
- Policy: rules deciding whether the action is allowed, denied, or requires approval.
- Decision: the evaluation outcome plus rationale (decision_outcome, matched rule, etc.).
- Receipt: the persisted record of the action request + decision + approval + execution + result/error.
- Claim: a lease granting exactly-one executor the right to perform the side effect for this receipt.
- Approval: explicit human/system approval required to execute certain actions.
- Tool: integration executor (http.request, github.issues.create, stripe.*).

==
4) The canonical flow (golden path)
==

This is the core of Authensor. Everything should map to this.

4.1 Evaluate (agent → control-plane)
1) Agent/tool runtime constructs an Action Envelope:
   - principal: who/what is acting (agent/service/user)
   - action: type + resource + parameters
   - context: environment, metadata
2) Agent calls POST /evaluate with envelope (and org/env headers).
3) control-plane:
   - loads active policy for org/env (or explicit allow-all fallback for alpha)
   - runs engine to compute decision

  - creates receipt row in Postgres (status/decision_outcome/approval_status/etc.)
4) Response includes:
  - receipt_id
  - receipt viewer link(s)
  - decision outcome and policy metadata

4.2 Execute (executor/MCP tool → claim → upstream → finalize receipt)
1) Executor requests claim:
  POST /receipts/:id/claim
  - If another claim exists: 409 with retry hints
  - If already executed/final: short-circuit (no upstream)
  - If not executable (denied/approval pending): 409 or not_executable response
2) After claim success:
  - perform the integration call with hardening + deterministic behavior
  - record attemptCount/duration + safe metadata
3) Finalize:
  PATCH /receipts/:id with claimId:
  - status=executed or failed
  - result/error normalized (no secrets)

4.3 Approvals (when decision_outcome requires approval)
- /evaluate records receipt with approval_status=pending
- No execution is allowed until approval_status=approved
- Approvals endpoints:
  POST /approvals/:receipt_id/approve
  POST /approvals/:receipt_id/reject
  POST /approvals/:receipt_id/expire
- State machine blocks “execute” while pending, and finalizes on reject/expire.

==========
5) Protocol conventions (action types, resources, context)
==========

5.1 action.type naming convention
- Use stable, namespace-like types:
  - http.request
  - github.issues.create / github.issues.comment / github.issues.list (etc)
  - stripe.charges.create / stripe.customers.create (etc)
- Keep action.type as the primary policy match axis.

5.2 resource convention
- Use URI-ish resource strings so policies can reason about scope:

  - http://… or https://…
  - github://repos/<owner>/<repo>/issues
  - stripe://customers/<id>/charges (etc)
- Policies should be able to constrain resource patterns as needed.

5.3 org/env scoping
- control-plane uses:
  - x-authensor-org (default: default)
  - x-authensor-env (default: dev)
- Policies are stored and “active” is selected per org/env.

==
6) Control-plane endpoints (alpha contract)
==

Stable endpoints (additive changes only; do not remove/rename in alpha):

Evaluation:
- POST /evaluate
  Returns: receipt_id, decision_outcome, policy {id, version, source}, receiptLink/receiptUrl, etc.

Receipts:
- GET /receipts/:id (JSON)
- GET /receipts/:id/view (HTML, safe, redacted, escaped, banner at top)
- GET /receipts/view (HTML list with filters and limit cap)
- POST /receipts/:id/claim (claimId + TTL; conflicts return retry hints)
- PATCH /receipts/:id (finalize executed/failed; requires claimId for execution states)

Approvals:
- POST /approvals/:id/approve
- POST /approvals/:id/reject
- POST /approvals/:id/expire

Policies:
- POST /policies (create versioned policy)
- POST /policies/active (set active pointer)
- GET /policies/active (read active policy for org/env)

Metrics:
- GET /metrics/summary?window=1h|24h|7d&org=...&env=...
  Returns: counts, ratios, and rule-based insights for alpha operations.

==
7) Receipt model (what matters; how to treat it)
==

Receipts are designed to answer:
- What did the agent want to do?
- What policy decided about it, and why?
- Was approval required and granted?
- Who executed it (claim), and what happened?
- Is it safe to retry?

7.1 Denormalized receipt columns (query fast)
- status (pending/executed/failed/skipped/cancelled)
- decision_outcome (allow/deny/require_approval/rate_limited/etc)
- tool_name, actor_id (principal)
- approval_status (pending/approved/rejected/expired)
- created_at/updated_at

7.2 JSON fields (auditable details)
- envelope, decision, approval, execution, result, error (as JSONB)

7.3 Deterministic execution fields
- claim_id / claimed_at / claim_expires_at
- execution_attempts
- claim events tracked in claim_events table (attempt/conflict/expired_reclaimed)

7.4 Viewer safety requirements (never regress)
- Recursive redaction: authorization/token/secret/password/cookie/set-cookie/api_key etc
- HTML escaping for ALL displayed values
- Proxy origin honoring ONLY when TRUST_PROXY=true
- Limit caps on list endpoints to avoid unbounded loads

==
8) Policy semantics (keep these stable)
==

- Policies are versioned objects, selected via “active pointer” per org/env.
- Engine semantics must be documented and covered by tests:
  - priority ordering
  - matching rules

  - defaultEffect behavior
  - glob matching / comparison operators

Alpha rule of thumb:
- First-match or highest-priority semantics MUST be explicit and tested.
- Any change to semantics is a contract-level change → CHANGELOG + tests update.

==========
9) Integration hardening standards (what “rock solid” means)
==========

9.1 HTTP tool (http.request)
- SSRF guard: block loopback/private/link-local/multicast/0.0.0.0 ranges
- DNS resolution checks; block unsafe resolved IPs
- Redirects blocked (manual)
- Optional plain HTTP gated by AUTHENSOR_ALLOW_HTTP (default false)
- Record resolved IPs safely; never record sensitive headers

Failure codes commonly seen:
- SSRF_BLOCKED
- REDIRECT_BLOCKED
- CONFIG_BLOCKED (if plain http disabled and requested)

9.2 GitHub tool
- Token only from env; must be redacted from receipts/logs
- allowlists:
  - empty allowlists: allow-all only in dev (with loud warning), deny-all in prod
- rate limiting:
  - 429 and 403 secondary rate limit mapped to RATE_LIMITED with retryAt
- Append dedupe marker: “Authensor-Receipt: <receiptId>” in create/comment bodies

9.3 Stripe tool
- Live gating:
  - prod requires AUTHENSOR_STRIPE_ALLOW_LIVE=true and STRIPE_LIVE_KEY
  - default false
- Bounds:
  - currency allowlist (default usd)
  - min/max amount (alpha defaults)
- Idempotency:
  - ALL writes must include Idempotency-Key=receiptId
- Deterministic retries:
  - only retry on network/5xx/429 with bounded attempts

9.4 Shared error model (no drift)
- Tool errors are normalized to stable codes:
  CONFIG_BLOCKED, INVALID_INPUT, RATE_LIMITED, UPSTREAM_4XX,
UPSTREAM_5XX,
  SECURITY_BLOCKED, TIMEOUT, plus tool-specific
SSRF_BLOCKED/REDIRECT_BLOCKED
- Errors must include retryable + retryAt when relevant
- Errors must never include secrets

==========
10) Metrics and “insights” (alpha ops loop)
==========

/metrics/summary is the alpha feedback loop.
It returns:
- grouped receipt counts (by_status, by_decision_outcome)
- claim friction counts (attempts, conflicts, expired_reclaimed)
- ratios (deny_rate, approval_required_rate, conflict_rate, etc)
- insights: rule-based hints when thresholds suggest something’s wrong

Insights are deterministic rules that help answer:
- deny spike -> policy too strict or wrong action.type
- CONFIG_BLOCKED spike -> missing env/allowlists/gating
- claim conflicts -> duplicate executors or aggressive retries
- expired reclaimed -> TTL too low or execution too slow
- approvals stuck -> approvals required but no approvals issued

Never turn this into “AI analytics” in alpha. Keep it explainable and actionable.

==========
11) Tooling, scripts, and expected dev workflow
==========

Canonical local workflow (alpha_onboarding.md is source):
- corepack enable
- pnpm install
- pnpm gen:check
- docker compose up -d postgres
- pnpm dev

- run examples/node-quickstart -> prints receiptLink
- open receipts viewer for debugging

Tooling standards:
- Prefer corepack-managed pnpm and avoid “nested pnpm calls” footguns.
- If turbo is used, ensure pnpm resolution works in scripts (avoid PATH reliance surprises).
- Keep tests green:
  - control-plane tests (pg-mem integration)
  - mcp-server tests (tool hardening)
  - engine tests (policy semantics)

==
12) How to add a new integration (beta path, but same standards)
==

Adding integration “X” must follow the alpha invariants:

Step-by-step:
1) Define action.type(s) and resource URI pattern.
2) Add/extend schemas if needed (packages/schemas) -> regenerate types.
3) Implement tool executor in packages/mcp-server:
   - MUST claim before upstream
   - MUST be safe-by-default with allowlists/gating
   - MUST normalize errors into shared error model
   - MUST store safe result subset only
4) Add unit tests for:
   - allowlist/gating
   - redaction (if relevant)
   - rate limit mapping (if relevant)
   - deterministic retry behavior
5) Add example usage in quickstart or MCP inspector section.
6) Update onboarding docs with required env vars + expected CONFIG_BLOCKED cases.
7) Update metrics if new outcomes become important (additive).

Definition of “rock solid integration”:
- deterministic (claim + idempotency where applicable)
- safe-by-default
- thoroughly tested
- debuggable via receipts viewer
- produces actionable metrics

==
13) Change control (avoid drift)
==

Alpha stability guarantees:
- Endpoints listed in alpha_onboarding.md are stable within 0.x (no silent breaking changes).
- Additive changes are allowed anytime:
  - new optional fields
  - new endpoints
  - new integrations/tools
- Breaking changes require:
  1) CHANGELOG entry
  2) schema or version bump
  3) migration notes if DB changes
  4) updated onboarding + tests

Rule: If it changes how a dev uses the system, it must be documented and tested.

==
14) Security reminders (do not violate)
==

- NEVER accept secrets via tool args.
- NEVER store secrets in receipts/logs/metrics/viewer.
- Keep redaction recursive and conservative.
- Always escape HTML in receipt viewer.
- Do not trust forwarded headers unless TRUST_PROXY=true.
- Default-deny dangerous capabilities (Stripe live, GitHub repos, HTTP plain).

==
15) “What’s next” (grounded roadmap)
==

Alpha (now):
- 3 hardened integrations, receipts viewer, approvals, metrics+insights, onboarding doc.

Beta (next):
- Expand to ~10 integrations (built on alpha learnings)

- Policy UI (basic web console)
- Real org/auth (multi-tenant auth, not just headers)
- Secrets manager integration (Vault / AWS Secrets Manager)
- Webhooks/notifications (approval required, deny spikes)
- Optional OpenMetrics/Prometheus export

==
Appendix A: The single question to ask when debugging
==

Open the receipt:
- What is decision_outcome?
- What is approval_status?
- What is claim state?
- What is error.code (if failed)?
Then follow the obvious action:
- deny -> fix policy/action.type match
- approval pending -> approve
- claim conflicts -> reduce concurrency/backoff
- CONFIG_BLOCKED -> set env/allowlists
- RATE_LIMITED -> wait until retryAt / reduce throughput

==
Appendix B: “If you change one thing, change it here”
==

- Contracts: packages/schemas/
- Semantics: packages/engine/
- Invariants: packages/control-plane (state machine/claims/approvals)
- Safety: packages/mcp-server (hardening + error normalization)
- DX: examples/ + receipt viewer + alpha_onboarding.md

End of master document.